

```
=====
# What is Deep Learning?
=====
```

- ** Deep Learning is a subset of Machine Learning.
- ** Deep Learning uses artificial neural networks to learn from large amounts of data.
- ** Deep Learning algorithms are inspired by the human brain.
- ** Deep Learning is part of Machine Learning, but it is more powerful for solving complex problems .

```
=====
# Machine Learning vs Deep Learning
=====
```

Machine Learning

- * Machine learning is a subset of AI
- * ML models learn from data using algorithms
- * Works well with small to medium data
- * Mostly works with structured data
- * Training time is usually faster
- * Easier to understand and explain

Deep Learning

- * Deep Learning is a subset of Machine Learning
- * DL uses neural networks with many layers
- * Needs a huge amount of data
- * Works well with unstructured data such as images, audio, video, and text
- * Training time is usually slower
- * Gives better accuracy for complex problems

```
=====
Real-life examples
=====
```

- 1) How does YouTube know which videos you may like?
- 2) How does Google Photos recognize a face?
- 3) How does ChatGPT understand our questions?
- 4) How does a self-driving car identify a person, road, or vehicle?

** All these systems deal with large amounts of data and need to identify complex patterns. This is where Deep Learning becomes important.

=====
Hierarchy
=====

Artificial Intelligence => Machine Learning => Deep Learning => Neural Networks

=====
Why do we need Deep Learning ?
=====

Q) Suppose we want to identify whether an image contains a cat or dog.

Traditional programming would require us to define rules: (This becomes extremely difficult)

IF ears are like this
AND eyes are like this
AND nose is like this
AND body is like this
THEN CAT

ELSE DOG

With Deep Learning:

Thousands of Images
 â†“
Neural Network
 â†“
Learns Patterns
 â†“
Cat / Dog

Note: We don't manually define all the rules. The neural network learns the patterns from data.

=====
ML Vs DL
=====

ML : Data Set ---> Feature Engineering ----> ML Algorithm ----> Prediction

DL : Data Set ---> Neural Networks ---> Automatically learn patterns ---> Prediction

Note: Traditional ML often depends heavily on manually selected features, while Deep Learning can learn useful representations automatically from raw or less-processed data.

=====
Where is Deep learning used ?
=====

1) Computer Vision

- Face recognition
- Object detection
- Medical image analysis
- Self driving cars

2) Natural Language processing

- Chat GPT
- Language Translation
- Text Classification
- Sentiment Analysis

3) Recommendation System

- Youtube
- Netflix
- Amazon
- Spotify

4) Speech

- Voice assistants
- Speech to text
- Voice Recognization

5) Generative AI

- Text Generation
- Image Generation
- Code Generation
- Audio Generation

=====
What is Neural Network ?
=====

Q) "How does Deep Learning actually learn?"

Ans) Through Neural networks

=> We have multiple layers in neural networks

- 1) Input Layer : Recieves input data
- 2) Hidden Layer : Learn patterns
- 3) Output Layer : Produces the final prediction

Ex :

Input
+
Hidden Layer 1
+
Hidden Layer 2
+
Hidden Layer 3

â†“
Output

Note : Multiple hidden layers make the network deep.

Deep Learning = Learning using deep/multi-layer neural networks.

=====
Deep Learning Model Execution Flow
=====

Input
â†“
Neural Network
â†“
Prediction
â†“
Compare with Actual Value
â†“
Calculate Loss
â†“
Backpropagation
â†“
Update Weights
â†“
Train Again

=====
Important Terminology
=====

1. Deep Learning
2. Neural Network
3. Neuron
4. Weight
5. Bias
6. Activation Function
7. Loss Function
8. Backpropagation
9. Optimizer
10. Epoch

Deep Learning

** Deep Learning is a subset of Machine Learning that uses Neural Networks with multiple layers to learn complex patterns from data.

** Deep Learning means teaching a computer to learn complex patterns using multiple layers of Neural Networks.

Note : Suppose we want to build a system that can identify whether an image contains a cat or dog.

We provide thousands of images:

Images

```
    â†“
Deep Learning Model
    â†“
Learns patterns
    â†“
Prediction
    â†“
Cat / Dog
```

The model may gradually learn:

```
Pixels
    â†“
Edges
    â†“
Shapes
    â†“
Eyes / Ears / Nose
    â†“
Face
    â†“
Cat or Dog
```

Neural Network

** A Neural Network is a computational model inspired by the structure of the human brain.

** It consists of interconnected neurons arranged in layers.

```
Input Layer
    â†“
Hidden Layer
    â†“
Hidden Layer
    â†“
Output Layer
```

Layers :

- 1) Input layer : Receives input data.
- 2) Hidden Layer(s) : Learns patterns from the input.
- 3) Output layer : Produces the final prediction.

** If a Neural Network has multiple hidden layers, it becomes a Deep Neural Network.

Neuron

** A Neuron is the basic processing unit of a Neural Network.

** You can think of a neuron as a small mathematical calculation unit.

A neuron:

```
-- Receives inputs
-- Applies weights
-- Adds bias
-- Calculates a value
-- Applies an activation function
-- Produces output
```

Basic Formula : $\text{Output} = \text{Activation Function}(\text{Input} \times \text{Weight} + \text{Bias})$

```
Input
â†“
Evaluate importance
â†“
Make calculation
â†“
Decision / Output
```

Weight

** Weight represents the importance of an input in a Neural Network.

Note : Suppose we want to predict whether a student will pass or fail.

```
inputs : study_hours, attendance, previous_marks
```

Maybe study hours are very important to this prediction.

```
Study Hours    â†’ Weight = 0.8
Attendance     â†’ Weight = 0.3
Previous Marks â†’ Weight = 0.5
```

Note :: Higher weight generally means the input has a stronger influence on the neuron's calculation.

Note: Weights are not manually decided by programmers in a typical Deep Learning training process. The model learns the weights from training data.

```
Random Weights
â†“
Prediction
â†“
Calculate Error
â†“
Update Weights
â†“
Better Prediction
```

Bias

** Bias is an additional value added to the weighted sum to help the Neural Network make better predictions.

Example :

```
Input = 5
Weight = 2
```

Then $\Rightarrow 5 \times 2 = 10$

```
if Bias = 3
```

```
10 + 3 = 13
```

```
Weighted Input + Bias
```

```
Weight â†’ Controls importance of input
```

```
Bias â†’ Helps shift/adjust the result
```

```
##### Activation Function #####
```

```
** An Activation Function determines whether and how strongly a neuron should be activated.
```

```
** It is applied after calculating:
```

```
    "Weighted Sum + Bias"
```

```
# Flow
```

```
Inputs
```

```
    â†’
```

```
Weights
```

```
    â†’
```

```
Weighted Sum
```

```
    â†’
```

```
Bias
```

```
    â†’
```

```
Activation Function
```

```
    â†’
```

```
Output
```

```
    Activation Function = Decision/Transformation function of a neuron.
```

```
##### Loss Function #####
```

```
** Now imagine the Neural Network makes a prediction.
```

```
    Actual Answer : Cat
```

```
    Model Prediction : Dog
```

```
** Obviously, the model made a mistake.
```

```
** We need to measure how wrong the prediction is, That is the job of the Loss Function.
```

```
** A Loss Function measures the difference between the actual output and the predicted output.
```

```
# Simple Example
```

```
Actual Value      = 100
```

```
Predicted Value  = 80
```

```
# The difference
100 - 80 = 20
```

```
##### Back propagation #####
```

```
** Backpropagation is the process of calculating how much each weight contributed to the error
```

```
** Improve the Neural Network behaviour based on back propagation
```

```
** Lets understand back propagation workflow step by step
```

```
    Step-1: Input data :: Input Data => Neural Network
```

```
    Step-2 : Prediction :: Neural Network => Prediction
```

```
    Step-3 : Calculate Loss :: Prediction => Compare with Actual Value => Loss
```

```
    Step-4 : Backpropagation :: The error information is propagated backward through the
network.
```

```
    Step-5 : Calculate Gradients :: Gradient tells the model how a small change in a weight
will affect the loss.
```

```
    Step-6 : Update Weights :: After calculating gradients, the optimizer updates the weights.
```

```
##### Optimizer #####
```

```
** An Optimizer is an algorithm that updates the weights and biases of a Neural Network to reduce
the loss.
```

```
Backpropagation => Calculates gradients
```

```
Optimizer => Uses gradients to update parameters
```

```
# Simple Calculation
```

```
Current Weight = 0.8
```

```
Gradient = 0.2
```

```
Learning Rate = 0.1
```

```
# New Update
```

```
New Weight = Old Weight - Learning Rate  $\times$  Gradient
```

```
New Weight = 0.8 - (0.1  $\times$  0.2)
```

```
New Weight = 0.78
```

```
##### Epoch #####
```

```
** An Epoch means one complete pass through the entire training dataset.
```

```
### suppose we have 10,000 training images
```

If the model processes all 10,000 images once: 1 Epoch

If the model processes all 10,000 images 2nd time: 2 Epochs

If the model processes all 10,000 images 3rd time: 3 Epochs

Summary

A Neural Network takes inputs, combines them using weights and bias, applies activation functions to produce a prediction, calculates the loss, uses backpropagation to calculate gradients, and an optimizer updates the weights to reduce the loss over multiple epochs.

=====
Assignment-1 : Develop Deep Learning Model to predict Student Pass or Fail
=====